



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad

B.Tech. in COMPUTER SCIENCE AND ENGINEERING

III Year I Semester Course Syllabus (KR24)

ARTIFICIAL INTELLIGENCE

Session 2: Deep Convolutional Generative Adversarial Networks (DCGANs)

Contents:

- 1. Introduction to DCGAN**
- 2. Why Replace Fully Connected Layers with Convolutional Layers?**
- 3. DCGAN Architecture: Generator and Discriminator**
- 4. Transposed Convolution in the Generator**
- 5. Progressive Image Generation Using the Generator**
- 6. Convolution and Down sampling in the Discriminator**
- 7. Common DCGAN Design Choices and Activation Functions**
- 8. DCGAN Training Process**
- 9. Basic GAN versus DCGAN**
- 10. Image Generation, Training Progression, and Analysis using Fashion-MNIST**

Prepared by:

Asha. M

Assistant Professor,

Department of CSE (AI & ML)

Keshav Memorial Institute of Technology

1. Learning objectives

- Explain the limitations of a basic GAN with fully connected layers.
- Describe the architecture of a Deep Convolutional GAN (DCGAN).
- Explain transposed convolution in the Generator.
- Explain convolution and downsampling in the Discriminator.
- Implement a DCGAN to generate synthetic images.
- Analyse generated images and identify common training problems.

2. Recap: What did we learn in the basic GAN?

In the previous class, we learned that a Generative Adversarial Network consists of two neural networks competing against each other.

Random Noise (Latent Vector z)

Example: 100 random numbers

Generator (G)

Creates a synthetic image

Discriminator (D)

Predicts whether an image is real or fake

Real image: Comes from the training dataset.

The Discriminator should classify it as real.

Fake image: Produced by the Generator.

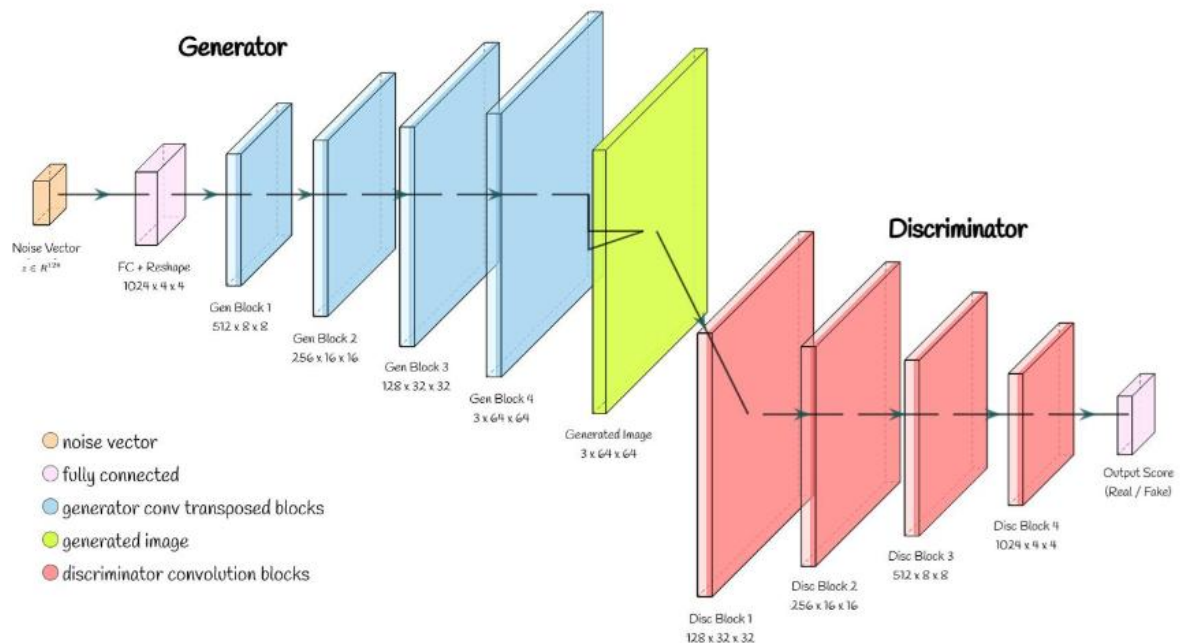
The Discriminator should initially classify it as fake.

3. What is DCGAN?

DCGAN stands for Deep Convolutional Generative Adversarial Network.

It is a GAN architecture that uses convolutional neural network (CNN) operations in the Generator and Discriminator instead of relying mainly on fully connected layers.

The original DCGAN architecture popularized a set of CNN design practices for stable GAN training and realistic image generation, especially for small images.



The fundamental idea is simple:

- The Generator transforms a compact latent vector into a full image.
- The Discriminator examines the image using convolutional filters and decides whether it resembles a real training image.
- Both networks improve through adversarial training.

DCGAN does not change the fundamental GAN objective. It changes how the two networks process image information.

4. Why replace fully connected layers with convolutional layers?

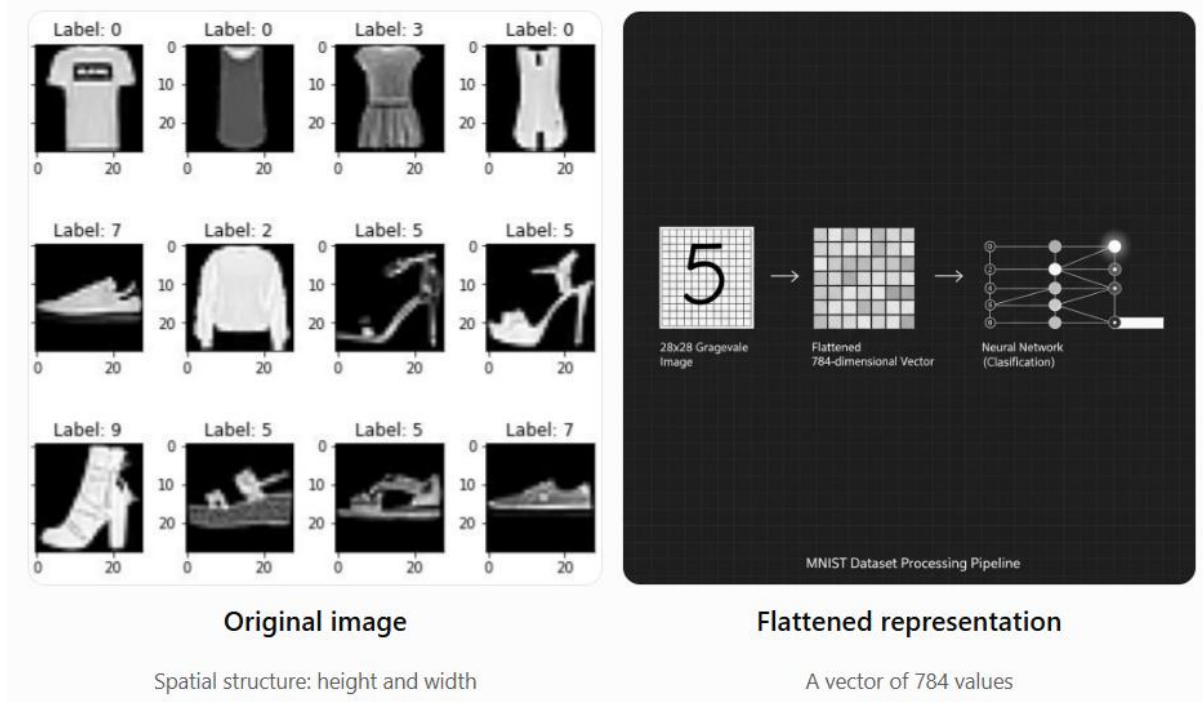
Let's understand this through an example.

Suppose we use an image of size 28×28 from the Fashion-MNIST dataset.

The image contains:

$$28 \times 28 = 784 \text{ pixel values}$$

In a fully connected network, we typically flatten the image into a vector of 784 values. Flattening makes it harder for the network to directly exploit the spatial arrangement of pixels.



Limitations of fully connected layers

1. Loss of explicit spatial structure

Flattening converts a two-dimensional image into a one-dimensional vector. Pixel positions still correspond to particular vector indices, but local neighbourhood relationships are no longer represented explicitly by the architecture.

2. Large number of parameters

Each input unit connects to every neuron in the next layer. For high-resolution images, this can become computationally expensive.

3. No built-in local feature detection

Convolutional layers are designed to learn local patterns such as edges, corners, textures, and shapes.

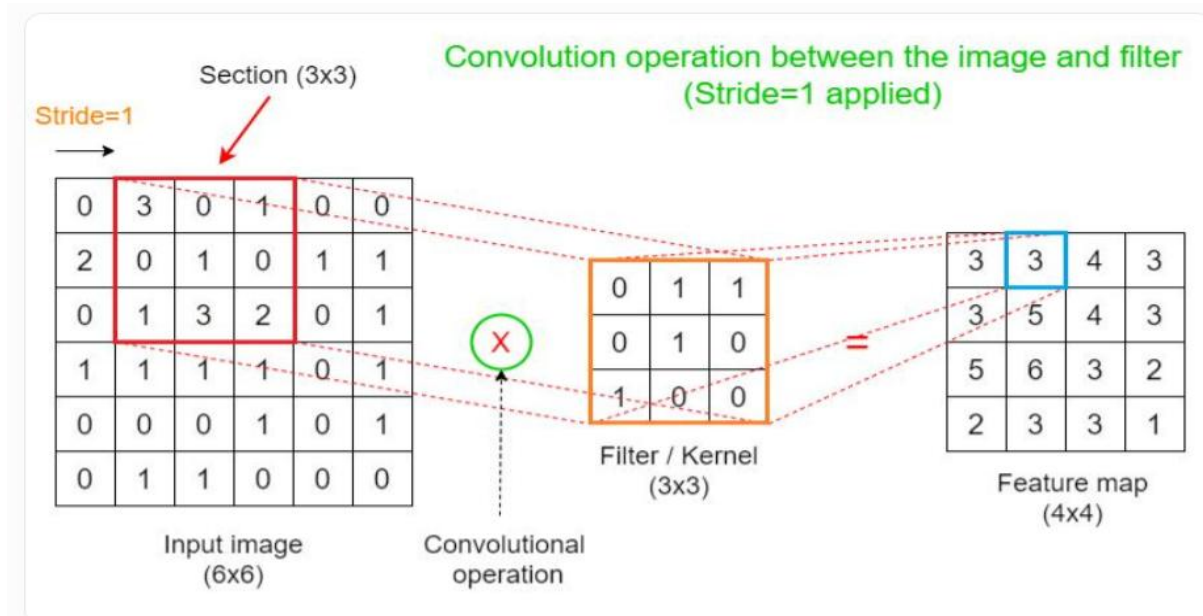
4. Difficulty scaling to complex images

Fully connected networks can generate images, but convolutional architectures are often more effective at learning spatially organized visual features.

Why convolution is useful

A convolutional layer applies a small learnable filter across different locations of an image.

For example, a filter might learn to detect a horizontal edge. The same filter can detect that edge whether it occurs at the top, middle, or bottom of an image.



The main advantages are:

Feature	Fully connected layer	Convolutional layer
Connections	Every input connects to every neuron	Each filter operates on local regions
Spatial structure	Not explicitly built into the architecture	Preserves spatial arrangement in feature maps
Parameters	Often large for images	Shared filter weights reduce parameter count
Feature learning	Learns general combinations of inputs	Learns local edges, textures and shapes
Suitability	General vector data	Image and spatial data

Convolution does not automatically guarantee better results, but it introduces a useful structural assumption for image generation and discrimination.

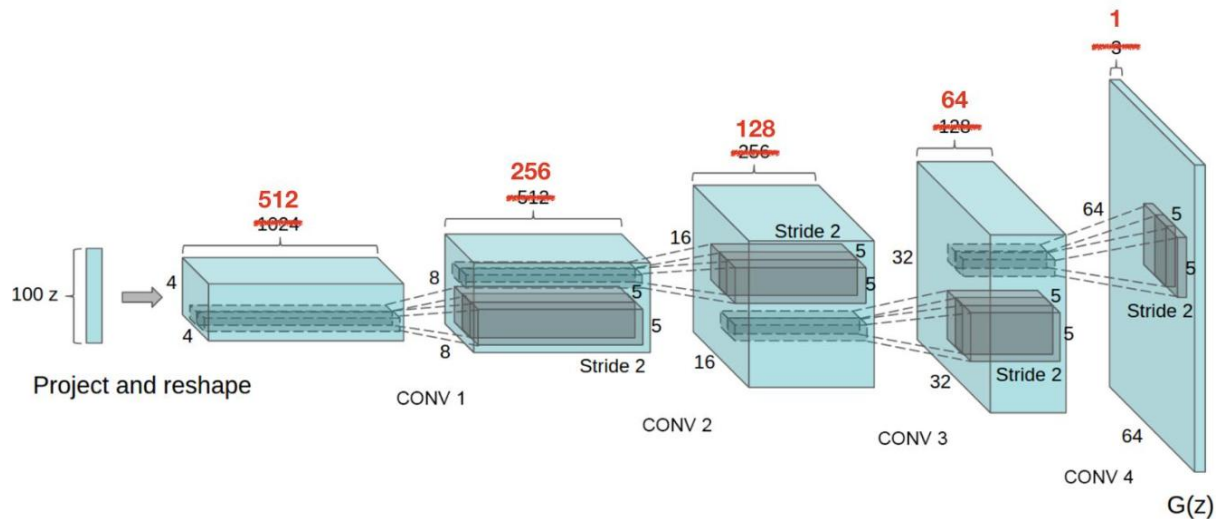
5. DCGAN architecture: Two networks working together

DCGAN has two major components. Let's examine them separately.

- A. Generator (G): The Generator starts with random noise and gradually transforms it into an image.

Typical flow

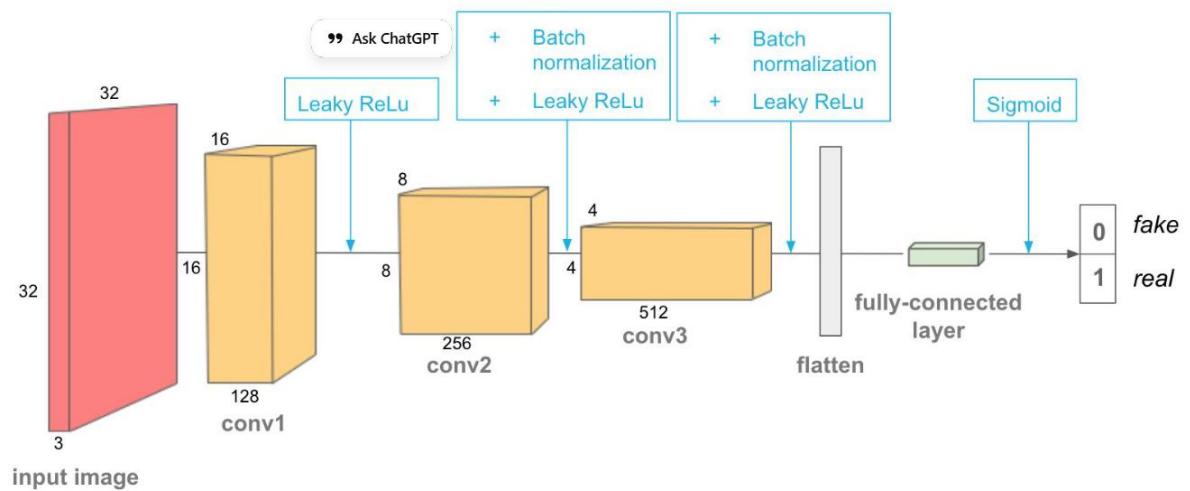
Random noise → Dense projection → Reshape → Transposed convolutions → Generated image



B. Discriminator (D): The Discriminator starts with an image and progressively extracts features to determine whether it is real or generated.

Typical flow

Input image → Convolutions → Downsampling → Feature extraction → Real/fake score



Remember this distinction:

- Generator: starts small and expands.
- Discriminator: starts with the complete image and compresses it into a decision.

6. Transposed convolution in the Generator

What is the problem?

Assume that the Generator starts with a latent vector of 100 random values.

$$z \in \mathbb{R}^{100}$$

A 100-dimensional vector is not yet an image. The Generator must transform it into a tensor representing image height, width, and channels.

For Fashion-MNIST, the final output should have dimensions:

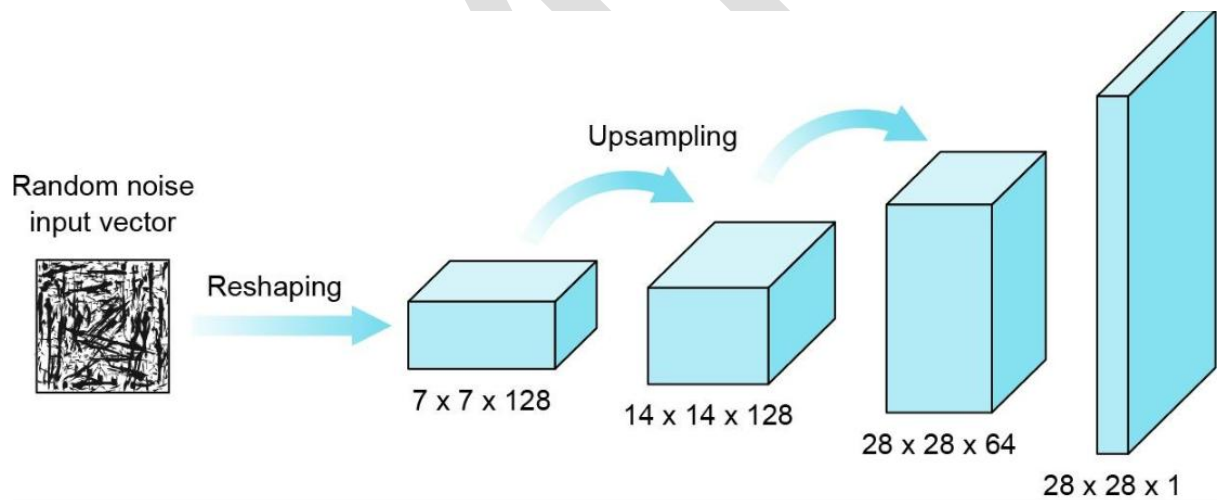
$$28 \times 28 \times 1$$

How do we expand a compact representation into an image? The Generator uses learned upsampling operations, commonly transposed convolutions.

What is transposed convolution?

A transposed convolution is a learnable operation that can increase the spatial dimensions of feature maps. It is commonly used in a Generator to progressively construct an image.

It is sometimes informally called deconvolution, but it is not generally the inverse of convolution.



Example: Progressive image generation

Consider a simplified Generator architecture.

Latent vector

100 random values

$$7 \times 7 \times 128$$

Initial feature map

$$14 \times 14 \times 64$$

Upsampling with transposed convolution

$$28 \times 28 \times 1$$

Generated grayscale image

These dimensions illustrate one possible design, not a required architecture. The channel count typically decreases as the spatial dimensions increase.

What does the Generator learn?

At different stages, the Generator can learn to construct increasingly meaningful visual features:

- Early stages: coarse spatial patterns and broad intensity distributions.
- Intermediate stages: edges, contours and local shapes.
- Later stages: finer details and textures.

These are intuitive interpretations of hierarchical feature learning, not strict rules about what every layer learns.

Common activation functions

- ReLU: Often used in intermediate Generator layers.
- Tanh: Commonly used in the final layer of a DCGAN Generator when training images are normalized to the range $[-1, 1]$.

Why Tanh? Because the output is constrained to a bounded range that matches the normalized training images.

7. Convolution and downsampling in the Discriminator

The Discriminator performs the opposite spatial transformation.

It receives either a real image from the dataset or a fake image from the Generator. It uses convolutional layers to extract features and progressively reduce spatial dimensions.

Example: Discriminator architecture

$$28 \times 28 \times 1$$

Input image

$$14 \times 14 \times 64$$

Convolution + downsampling

$$7 \times 7 \times 128$$

Deeper feature extraction

1 output score

Real or fake assessment

Again, these are illustrative dimensions. The exact architecture depends on the input image size and the chosen convolution parameters.

What is downsampling?

Downsampling reduces the height and width of a feature map. In DCGAN, this is typically achieved using convolutional layers with a stride greater than one.

For example:

$$28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$$

As the spatial dimensions decrease, the network can learn to represent increasingly broad patterns rather than processing every pixel independently.

Why does the Discriminator need this?

Imagine the Generator produces an image that looks vaguely like a shoe.

The Discriminator may learn to examine:

- Whether edges resemble a shoe outline.
- Whether the sole and upper regions have plausible shapes.
- Whether textures and local patterns resemble real training images.
- Whether the overall combination of features resembles the real dataset.

The Discriminator does not necessarily identify these objects in human terms. It learns statistical features that help distinguish real training images from generated images.

Common DCGAN design choices

Component	Typical choice	Purpose
Discriminator convolution	Stride 2	Downsample feature maps
Discriminator activation	LeakyReLU	Maintain gradient flow for negative inputs
Generator intermediate layers	ReLU	Learn nonlinear transformations
Generator final layer	Tanh	Produce normalized pixel values
Normalization	Batch normalization in many intermediate layers	Help stabilize training
Pooling	Often avoided in favour of strided convolutions	Learn downsampling through trainable filters

These are common DCGAN practices rather than mandatory rules for every GAN.

8. How does DCGAN training work?

The overall adversarial learning process remains the same as in the basic GAN.

Step 1: Load real images

Sample a batch of real Fashion-MNIST images from the dataset.

Step 2: Generate fake images

Sample random latent vectors and pass them through the Generator.

Step 3: Train the Discriminator

Encourage it to assign higher realness scores to real images and lower scores to fake images.

Step 4: Train the Generator

Update the Generator so that its fake images receive higher realness scores from the Discriminator.

Step 5: Repeat

Continue training over many batches and epochs while monitoring generated images and training behaviour.

The mathematical objective

The original GAN minimax objective can be written as:

$$\min_G \max_D V(D, G)$$

where

$$V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]$$

Here:

- x is a real image.
- z is a random latent vector.
- $G(z)$ is the generated image.
- $D(x)$ is the Discriminator's realness score.
- p_{data} represents the real data distribution.
- p_z represents the distribution from which random noise is sampled.

The Discriminator tries to distinguish real images from generated images, while the Generator tries to produce images that the Discriminator classifies as real.

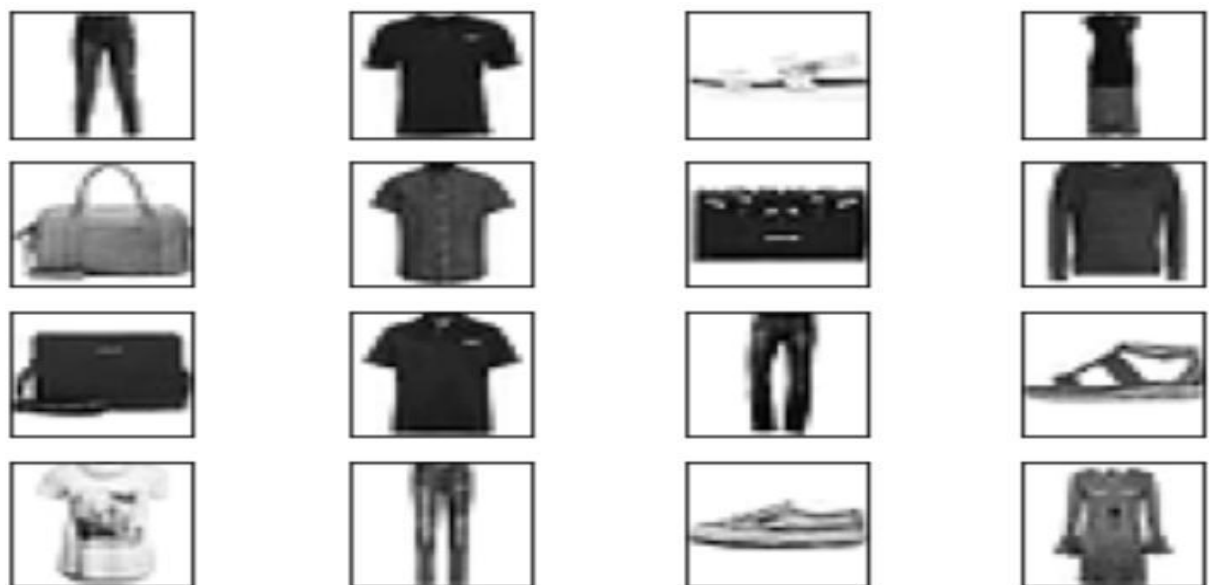
In practical implementations, the Generator is often trained using a non-saturating loss to improve gradient behaviour.

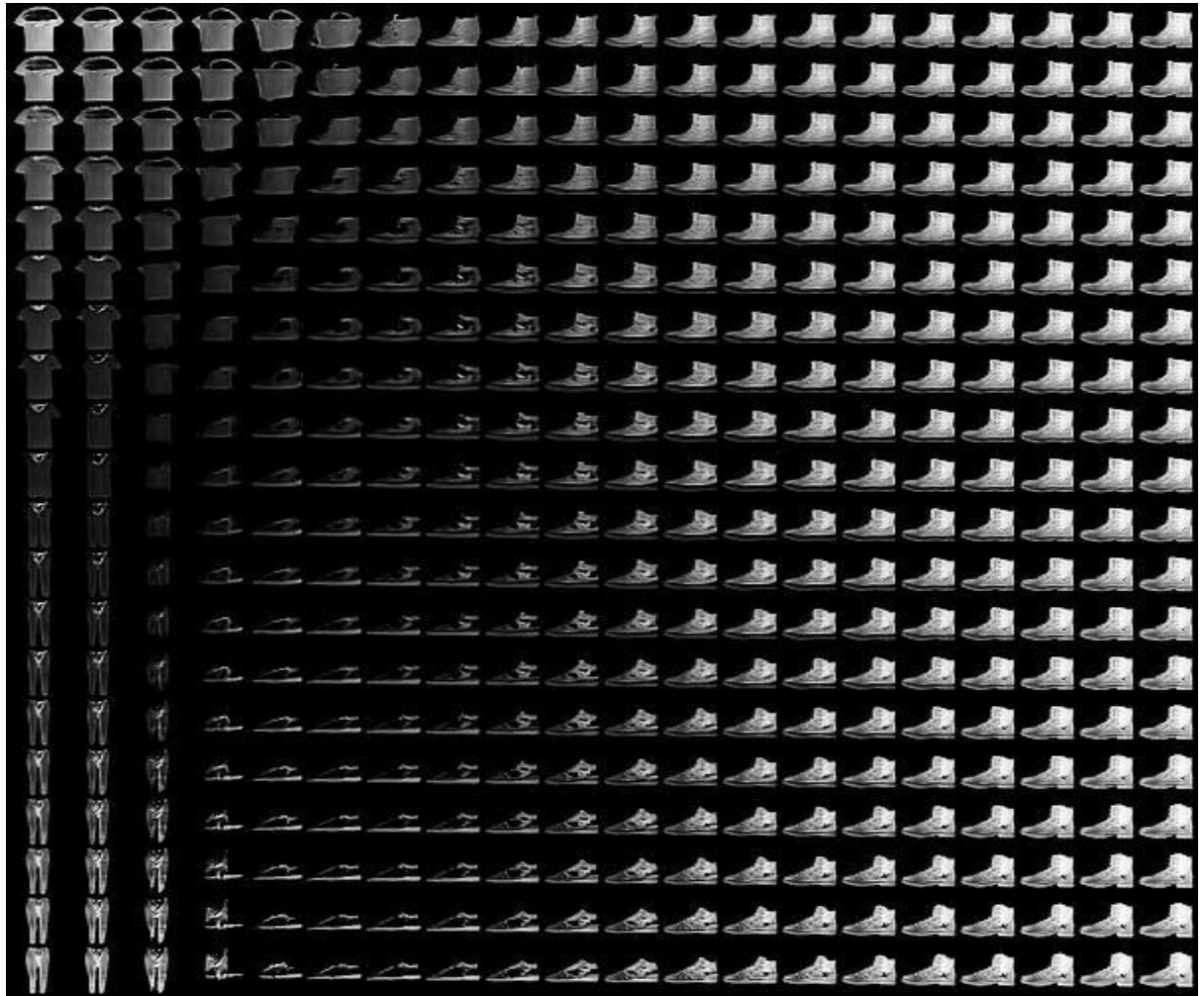
9. Basic GAN versus DCGAN

Feature	Basic GAN	DCGAN
Typical architecture	Fully connected networks	Convolutional networks
Image representation	Often flattened	Spatial feature maps
Generator	Dense layers may directly produce pixels	Commonly uses transposed convolutions
Discriminator	Dense layers process flattened pixels	Convolutions extract visual features
Spatial structure	Not explicitly built into the architecture	Incorporated through convolutional operations
Typical use in teaching	Introduce adversarial learning	Learn CNN-based image generation
Main challenge	Scaling to complex spatial data	Training instability and artifacts remain possible

Important: DCGAN is not guaranteed to generate more realistic images in every situation. Dataset quality, network design, hyperparameters, training duration, and stability all affect results.

10. Why do DCGAN-generated images become more structured?





Initially, the Generator produces images that may look like random noise. Through training, it gradually learns patterns that resemble the training distribution.

For example, when trained on Fashion-MNIST, the Generator may learn patterns corresponding to clothing outlines, footwear shapes, bags, and other visual structures.

The progression can be understood as follows:

- Early training: Random patterns, unclear shapes, and noise.
- Intermediate training: Some recognizable outlines and object-like structures.
- Later training: More coherent shapes and textures, if training is successful.

However, longer training does not always mean better images. Mode collapse, unstable gradients, or an overly powerful Discriminator may prevent improvement.